Kyle Stanford

Parallel Systems CS380P

Dr. Lin

21 October 2026

LAB 3 REPORT

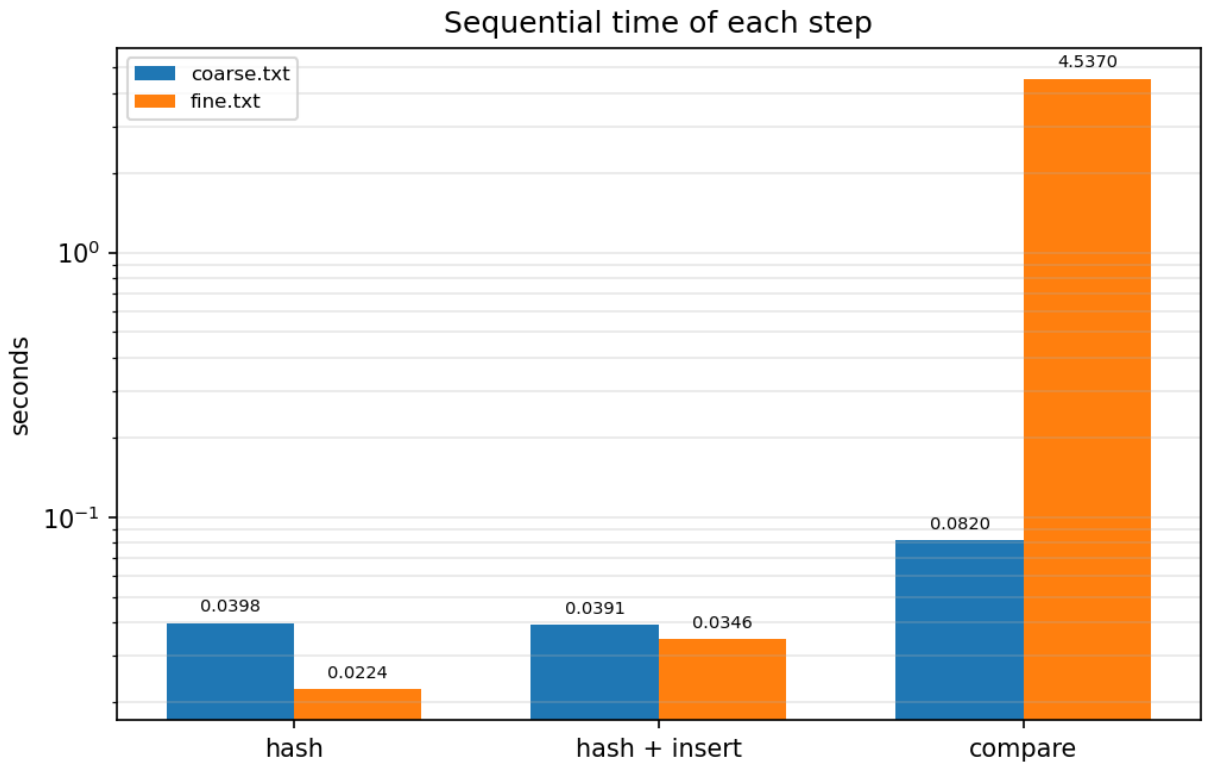
Introduction

This lab checks whether binary search trees hold the same values, and it asks for the work to be parallelized in stages. The program reads one tree per line, takes each tree's id from its line number, and inserts the values in order. Each tree's in-order traversal is hashed into a number from 0 to 999, and only trees with the same hash are compared. Everything lives in one file, BST_opt.go, with separate worker flags for each stage. I ran each configuration five times and report the mean.

Hardware

All measurements were taken on a Codio VM with 8 cores of an AMD EPYC 7571. The VM is shared with other tenants, so the coarse numbers in particular are noisy.

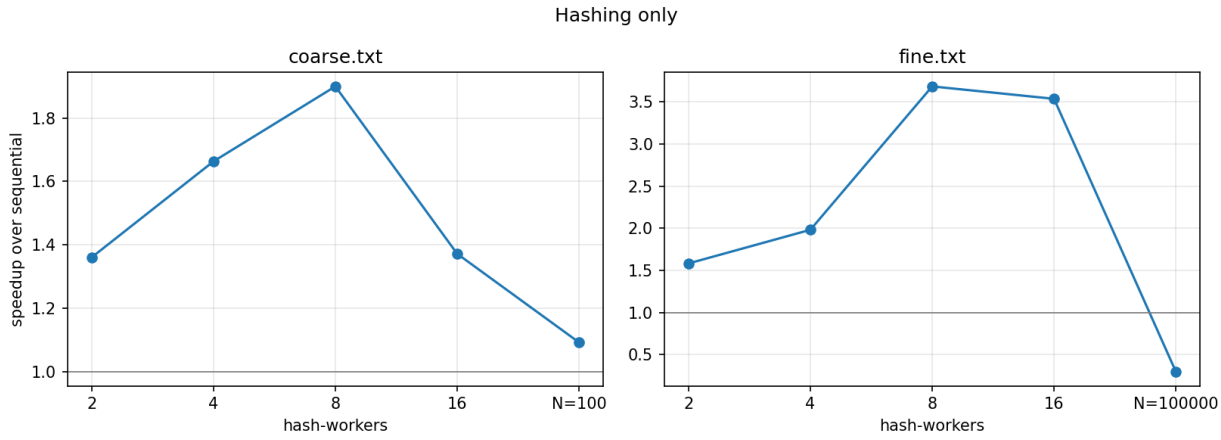
Step 1: Sequential solution



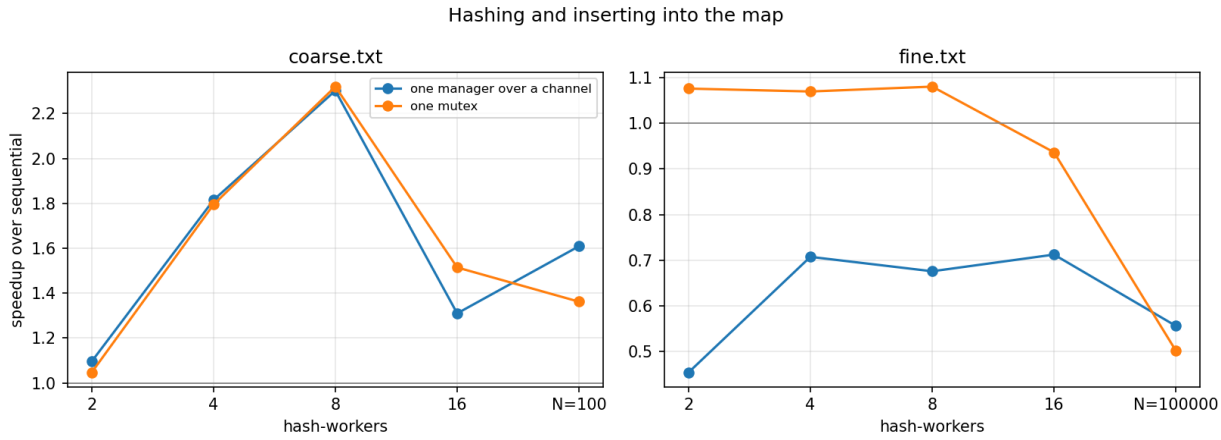
On coarse.txt, which has 100 trees of 10000 values, hashing takes 0.0398 seconds, hashing plus insertion takes 0.0391 seconds, and tree comparison takes 0.0820 seconds. On fine.txt, which has 100000 trees of 10 values, hashing takes 0.0224 seconds, hashing plus insertion takes 0.0346 seconds, and comparison takes 4.537 seconds.

Step 2: Parallelize hash operations

I parallelized hashing with a set of hash workers, each owning one contiguous block of tree ids. With one data worker, the hash workers send (hash, id) pairs over a channel to a single manager goroutine that owns the map. With data workers equal to hash workers, each worker inserts directly into a shared map guarded by one mutex.



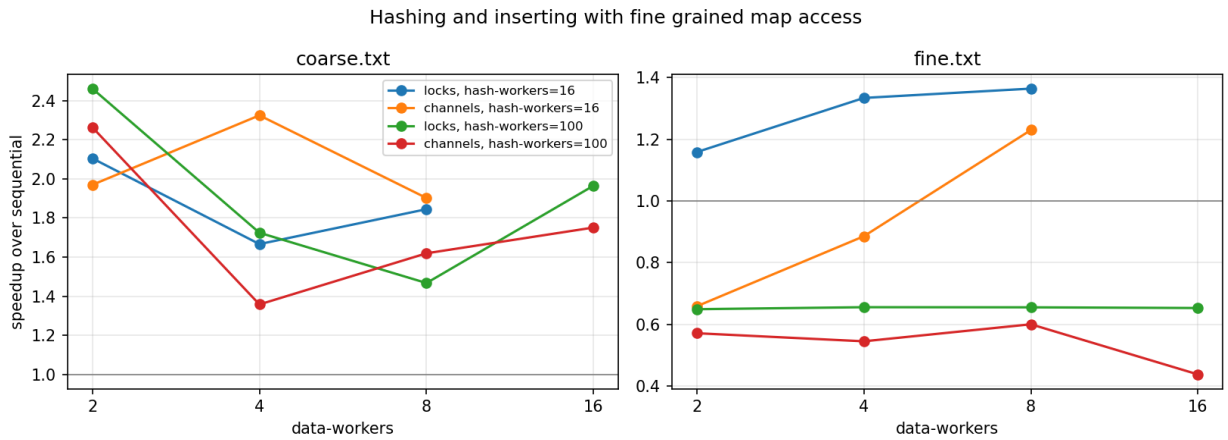
Hashing alone scaled to 1.90x on coarse and 3.68x on fine at 8 workers. With insertion added, coarse reached 2.30x with the channel and 2.32x with the lock at 8 workers, a little above hashing alone, which I put down to run to run noise on the shared VM. On fine the lock stayed near 1.08x, while the channel stayed between 0.45x and 0.71x, slower than sequential.



On coarse neither approach was significantly faster. The two were within noise because there are only 100 inserts against 40 milliseconds of hashing. On fine the lock was faster at every worker count, and the channel had more overhead: each of the 100000 sends passes through the manager goroutine and the scheduler, while an uncontended mutex is close to one atomic operation.

Fine.txt hurt scalability because each tree is tiny, so one synchronization costs about as much as the work it protects. Insertion also adds about 12 milliseconds of serialized work on top of 22 milliseconds of hashing.

For extra credit I split the map into j shards by hash, each behind its own mutex or its own manager goroutine. On fine with 16 hash workers this helped: sharded locks went from 0.94x with one mutex to 1.36x with 8 shards, and sharded channels from 0.71x to 1.23x. So access to the single map was a bottleneck there. On coarse it made no difference, since inserts were never the cost. Locks beat channels on fine, so the lock version is the default.



Spawning hash-workers goroutines beat one goroutine per BST on both inputs. On fine, 8 workers reached 3.68x against 0.30x for one goroutine per tree, about 12 times faster, and on coarse it was 1.90x against 1.09x. The goroutine per tree version on fine ran more than 3 times slower than sequential, which surprised me, but starting a goroutine costs more than hashing a 10 node tree. So Go does not make the thread count irrelevant: with tasks this small the count has to be bounded, and a pool sized near the core count was fastest on both inputs.

My preference is the mutex version. It was faster on fine and much simpler, since it is a lock around an append rather than a manager goroutine that has to be drained and closed.

Step 3: Parallelize tree comparisons

Only trees with the same hash are compared, and each hash group gets its own small adjacency matrix rather than one N by N matrix. Comparisons run either one goroutine each, or through a pool fed by a bounded buffer built from a mutex and two condition variables.



Coarse.txt needs 2646 comparisons, which take 0.0840 seconds sequentially. One goroutine per comparison reached 2.35x. The pool reached 1.62x with 2 workers, 2.73x with 4, 2.51x with 8 and 3.01x with 16. I did not run fine.txt comparisons in parallel.

Neither approach was dramatically faster. The pool won at 16 workers, 3.01x against 2.35x, but each comparison takes about 32 microseconds, so starting a goroutine per pair costs little. The per-comparison version still puts all 2646 goroutines on the scheduler at once, while the pool keeps a fixed set busy.

The goroutine per comparison version is about 8 lines. The buffer plus pool is about 60, with the mutex, condition variables and shutdown logic, and it is much easier to get wrong. On coarse that extra complexity was only marginally worth it.

I liked Go. Goroutines, channels and WaitGroup were pleasant and quick to write. I'd still pick C++ when I need the last bit of speed, since the garbage collector and lack of control over memory layout get in the way, but Go was much faster to write.

Time spent

Approximately [FILL IN] hours.